

Tema 3.

Preprocesamiento y análisis morfosintáctico con spaCy

Pandas para lingüistas: organización, análisis y exportación de datos

Material complementario | Programación para PLN | Universidad Pontificia Comillas

1. Introducción: por qué aparece pandas en una práctica de PLN

En una práctica de Procesamiento del Lenguaje Natural, el primer objetivo suele ser analizar lingüísticamente un texto: tokenizarlo, eliminar *stopwords*, lematizarlo, etiquetar sus categorías gramaticales o extraer sustantivos relevantes. Para estas tareas, bibliotecas como spaCy o NLTK resultan fundamentales. Sin embargo, una vez que el texto ha sido procesado, aparece una segunda necesidad: organizar los resultados en una estructura clara, consultable y exportable.

Ahí es donde entra **pandas**.

Pandas es una biblioteca de Python diseñada para trabajar con datos estructurados o etiquetados, especialmente en forma de tablas. Su estructura más importante es el DataFrame, que puede entenderse como una tabla similar a una hoja de cálculo, con filas y columnas. Esta forma de organización resulta especialmente útil en PLN porque muchos resultados lingüísticos pueden representarse de manera tabular: cada fila puede corresponder a un token, una oración, un documento o una ocurrencia léxica; cada columna puede almacenar información como el token original, el lema, la categoría gramatical, la frecuencia o el número de documento. La propia documentación de pandas describe la biblioteca como una herramienta para trabajar con datos relacionales o etiquetados de forma flexible y expresiva.

Así, Pandas no analiza lingüísticamente el texto, sino que permite guardar, ordenar, inspeccionar y exportar el resultado del análisis. Esta distinción es importante: spaCy produce anotaciones lingüísticas; pandas ayuda a convertir esas anotaciones en una tabla reutilizable.

2. De un texto a una tabla: el cambio conceptual

Para un lingüista, el punto de partida habitual es un texto continuo: La traducción automática neuronal ha transformado los sistemas de traducción.

Después de procesarlo con spaCy, ese texto se convierte en una secuencia de objetos Token. Cada token contiene información lingüística:

```
token.text      # forma superficial
token.lemma_    # lema
token.pos_      # categoría gramatical
token.is_alpha  # si está compuesto solo por letras
token.is_stop   # si spaCy lo considera stopwords
```

El problema es que esta información, aunque esté disponible en Python, todavía no está organizada en una forma cómoda para su revisión. Por ejemplo, una lista de tuplas como esta:

```
[
    ("traducción", "traducción", "NOUN"),
    ("automática", "automático", "ADJ"),
    ("neuronal", "neuronal", "ADJ")
]
```

es útil para programar, pero menos clara para revisar, corregir, exportar o entregar como resultado de una práctica. Pandas permite transformar esa lista en una tabla:

token	lema	pos
traducción	traducción	NOUN

token	lema	pos
automática	automático	ADJ
neuronal	neuronal	ADJ

Este paso es metodológicamente relevante. En lingüística de corpus y PLN, una parte importante del trabajo consiste en pasar de datos no estructurados —textos— a datos estructurados —tablas, matrices, índices o corpus anotados—. Esta idea conecta con el principio de los datos ordenados o *tidy data*: cada variable debe ocupar una columna, cada observación una fila y cada tipo de unidad observacional una tabla (Wickham, 2014).

3. ¿Qué es un DataFrame?

Un DataFrame es la estructura central de pandas. Puede entenderse como una tabla bidimensional con:

- **filas**, que representan observaciones;
- **columnas**, que representan variables;
- **índices**, que identifican las filas;
- **nombres de columnas**, que identifican las variables.

En un análisis lingüístico, una fila puede representar una unidad textual. La elección de esa unidad depende del objetivo del análisis.

Unidad de análisis	Qué representa cada fila	Ejemplo de columnas
Token	Cada palabra o signo procesado	token, lema, pos, is_stop
Lema	Cada lema distinto	lema, frecuencia, pos_dominante
Oración	Cada oración del texto	oracion, num_tokens, num_sustantivos
Documento	Cada texto de un corpus	id_doc, titulo, num_tokens, tema
Término	Cada candidato terminológico	termino, frecuencia, categoria

Esta flexibilidad es una de las razones por las que pandas se ha convertido en una herramienta central en ciencia de datos con Python. En su artículo fundacional, McKinney (2010) presentó pandas como una biblioteca orientada a resolver problemas prácticos de manipulación y análisis de datos estructurados en Python.

4. Pandas y PLN: qué aporta realmente

En una práctica de PLN, pandas aporta al menos cinco ventajas.

4.1. Permite visualizar los resultados con claridad

Cuando trabajamos solo con listas, los datos pueden volverse difíciles de leer:

```
tokens = ["traducción", "automática", "neuronal"]
lemas = ["traducción", "automático", "neuronal"]
pos = ["NOUN", "ADJ", "ADJ"]
```

Con pandas, esos datos se integran en una tabla:

```
import pandas as pd
```

```
df = pd.DataFrame({  
    "token": tokens,  
    "lema": lemas,  
    "pos": pos  
})
```

El resultado es más interpretable para un perfil lingüístico, porque se parece a una tabla de anotación lingüística.

4.2. Facilita la revisión lingüística

Un lingüista puede querer comprobar si los lemas son correctos, si una palabra ha sido etiquetada correctamente como sustantivo o si el sistema ha cometido errores de análisis. Un DataFrame permite ver esa información fila por fila.

Por ejemplo:

`df.head(10)` → muestra las diez primeras filas de la tabla.

`df[df["pos"] == "NOUN"]` → muestra solo los sustantivos.

`df[df["lema"] == "traducción"]` → muestra todas las ocurrencias cuyo lema es traducción.

Estas operaciones son muy útiles para pasar del procesamiento automático a la interpretación lingüística.

4.3. Permite calcular frecuencias

Las frecuencias son una operación clásica en lingüística de corpus. Pandas permite calcularlas de forma sencilla: `df["lema"].value_counts()`

También se pueden calcular frecuencias por categoría gramatical: `df["pos"].value_counts()`

O frecuencias de sustantivos: `df[df["pos"] == "NOUN"]["lema"].value_counts()`

Esto conecta directamente con tareas de PLN como la extracción de vocabulario relevante, la identificación de términos frecuentes o la comparación léxica entre textos.

4.4. Facilita la exportación

Una vez organizada la información, pandas permite guardarla en un archivo externo. En el ejercicio 5, el formato elegido es CSV. La documentación oficial de pandas describe `DataFrame.to_csv()` como el método que permite escribir un DataFrame en un archivo separado por comas.

Por ejemplo: `df.to_csv("tokens_lemmatizados.csv", index=False, encoding="utf-8")`

Este archivo puede abrirse después con Excel, LibreOffice Calc, Google Sheets, R, Python u otras herramientas de análisis.

4.5. Favorece la reproducibilidad

En investigación y docencia en PLN, no basta con obtener un resultado en pantalla. Es importante que los datos procesados puedan guardarse, compartirse y revisarse. La reproducibilidad es una preocupación central en el PLN contemporáneo, especialmente cuando los experimentos dependen de decisiones de preprocesamiento, versiones de librerías, modelos lingüísticos y criterios de anotación (Belz *et al.*, 2021).

Guardar los resultados en CSV permite documentar qué tokens fueron conservados, qué lemas se obtuvieron, qué etiquetas gramaticales asignó el modelo y qué datos se usaron después para calcular frecuencias o construir un glosario.

5. ¿Qué es un archivo CSV?

CSV significa *Comma-Separated Values*, es decir, «valores separados por comas». Es un formato de archivo de texto plano que representa datos tabulares. Un CSV puede verse así:

```
token, lema, pos
traducción, traducción, NOUN
automática, automático, ADJ
neuronal, neuronal, ADJ
```

Aunque se llama «separado por comas», en algunos contextos también puede usarse punto y coma como separador. Esto ocurre a menudo en configuraciones regionales donde la coma se emplea como separador decimal. Pandas permite controlar este aspecto mediante el parámetro `sep`. Por ejemplo:

```
df.to_csv("tokens_lemmatizados.csv", index=False, encoding="utf-8", sep=";")
```

Lo importante es entender que un CSV no es exactamente una hoja de cálculo, sino un archivo de texto estructurado que muchas aplicaciones pueden leer como tabla.

6. Cómo usar pandas para analizar los resultados lingüísticos

Una vez creado el DataFrame, no solo podemos exportarlo. También podemos hacer análisis sencillos.

Acción	Código	Explicación
Ver las categorías gramaticales más frecuentes	<code>df["pos"].value_counts()</code>	Esto permite saber cuántos sustantivos, verbos, adjetivos o determinantes aparecen en el texto.
Extraer solo los sustantivos	<code>sustantivos = df[df["pos"] == "NOUN"]</code>	Ahora sustantivos es otro DataFrame, pero solo contiene las filas cuya categoría gramatical es NOUN.
Calcular los lemas sustantivos más frecuentes	<code>sustantivos["lema"].value_counts()</code>	Esta operación puede servir para crear un glosario preliminar.
Guardar solo los sustantivos	<code>sustantivos.to_csv("sustantivos.csv", index=False, encoding="utf-8")</code>	Esto genera un archivo específico con los sustantivos detectados.
Ordenar la tabla	<code>df.sort_values(by="lema")</code>	Ordena las filas alfabéticamente por lema.
Eliminar duplicados	<code>df[["lema", "pos"]].drop_duplicates()</code>	Permite obtener una lista de lemas únicos con su categoría gramatical.

7. Diferencia entre una lista, un diccionario y un DataFrame

Para estudiantes de lingüística, puede ser útil comparar tres formas de representar la misma información.

Lista

```
tokens = ["traducción", "automática", "neuronal"]
```

Una lista guarda elementos en orden, pero no dice qué representa cada elemento más allá de su posición.

Diccionario

```
token = {  
    "token": "traducción",  
    "lema": "traducción",  
    "pos": "NOUN"  
}
```

Un diccionario permite asociar nombres a valores. Es adecuado para representar la información de una unidad lingüística.

Lista de diccionarios

```
datos = [  
    {"token": "traducción", "lema": "traducción", "pos": "NOUN"},  
    {"token": "automática", "lema": "automático", "pos": "ADJ"},  
    {"token": "neuronal", "lema": "neuronal", "pos": "ADJ"}  
]
```

Una lista de diccionarios permite representar muchas unidades lingüísticas.

DataFrame

```
df = pd.DataFrame(datos)
```

El DataFrame convierte esa estructura en una tabla analizable y exportable.

8. Pandas como puente entre programación y análisis lingüístico

Para alumnado procedente de lingüística, pandas puede entenderse como un puente entre dos mundos:

1. El mundo lingüístico, donde se interpretan formas, lemas, categorías, construcciones, frecuencias y patrones.
2. El mundo computacional, donde los datos deben representarse en estructuras manipulables por código.

Pandas no exige abandonar la mirada lingüística. Al contrario, permite organizarla mejor. Una tabla de pandas puede representar una anotación lingüística tradicional, pero con la ventaja de que puede filtrarse, ordenarse, agruparse, exportarse y analizarse automáticamente.

15. Buenas prácticas al usar pandas en prácticas de PLN

Para que el uso de pandas sea claro y reproducible, conviene seguir algunas buenas prácticas.

Recomendación	Explicación
Usar nombres de columnas transparentes	Mejor: "token", "lema", "pos" que: "t", "l", "p" Los nombres claros ayudan a leer el código y facilitan la revisión.
Guardar siempre con encoding="utf-8"	Especialmente en español y en corpus multilingües. df.to_csv("archivo.csv", index=False, encoding="utf-8")
Evitar guardar el índice salvo que tenga significado	En la mayoría de prácticas introductorias: index=False es la opción adecuada.
Separar análisis lingüístico y organización tabular	Primero se procesa el texto: doc = nlp(texto) Después se extraen los datos: datos.append(...) Finalmente se crea la tabla: df = pd.DataFrame(datos) Esta separación ayuda a entender qué parte del código hace análisis lingüístico y qué parte organiza los resultados.
Documentar qué representa cada fila	En un comentario o celda Markdown, conviene indicar: <i>Cada fila de la tabla representa un token alfabético del texto.</i> O bien: <i>Cada fila representa un lema sustantivo y su frecuencia.</i> Esto evita ambigüedades metodológicas.

18. Conclusión

Pandas es una herramienta especialmente útil en PLN porque permite transformar los resultados del análisis lingüístico en datos tabulares. Para estudiantes de lingüística, su utilidad principal no está en sustituir la interpretación lingüística, sino en hacerla más organizada, visible y reutilizable.

En el contexto del ejercicio 5, pandas cumple una función concreta: convertir la información generada por spaCy —tokens, lemas y categorías gramaticales— en una tabla que puede guardarse como CSV. Esta exportación permite revisar los resultados fuera del cuaderno, compartirlos, abrirllos en una hoja de cálculo o utilizarlos en análisis posteriores.

La idea fundamental es la siguiente: spaCy analiza lingüísticamente el texto; pandas organiza y exporta los resultados de ese análisis.

Esta distinción ayuda a comprender por qué pandas aparece al final del ejercicio: no forma parte del análisis morfosintáctico propiamente dicho, sino de la fase de estructuración, documentación y conservación de los datos obtenidos.

Referencias

Arnold, T. (2017). A tidy data model for natural language processing using cleanNLP. *The R Journal*, 9(2), 1–20.
<https://doi.org/10.32614/RJ-2017-035>

- Belz, A., Agarwal, S., Shimorina, A. y Reiter, E. (2021). A systematic review of reproducibility research in natural language processing. *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, 381–393. <https://doi.org/10.18653/v1/2021.eacl-main.29>
- Jurafsky, D. y Martin, J. H. (2026). *Speech and language processing: An introduction to natural language processing, computational linguistics, and speech recognition with language models* (3rd ed., online manuscript released January 6, 2026). Stanford University. <https://web.stanford.edu/~jurafsky/slp3/>
- McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 56–61. <https://doi.org/10.25080/Majora-92bf1922-00a>
- The pandas development team. (2026a). *10 minutes to pandas*. pandas documentation. https://pandas.pydata.org/docs/user_guide/10min.html
- The pandas development team. (2026b). *pandas.DataFrame*. pandas documentation. <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.html>
- The pandas development team. (2026c). *pandas.DataFrame.to_csv*. pandas documentation. https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.to_csv.html
- Wickham, H. (2014). Tidy data. *Journal of Statistical Software*, 59(10), 1–23. <https://doi.org/10.18637/jss.v059.i10>